

Python

Fundamentals for the AI-Native Builder

Language, ecosystem, backend, data, and AI/ML — for non-developers shipping with AI

The language behind most AI, most data work,
and a great many backends.
What you need to direct AI agents writing it.

Contents

Introduction	6
Who this is for.....	6
How this relates to the React & TypeScript doc	6
Why Python matters in the AI era	6
How to use this document	7
Part 1 — The Python Language Essentials	8
1.1 The shape of Python	8
1.2 Variables and basic types	8
1.3 Strings and f-strings.....	8
1.4 Lists, tuples, dicts, sets.....	9
1.5 Slicing.....	9
1.6 Control flow	10
1.7 Comprehensions	10
1.8 Functions.....	11
Mutable default arguments — the classic trap.....	11
1.9 Iteration and the iterator protocol	11
1.10 Generators — lazy sequences.....	12
1.11 Exceptions.....	12
1.12 Context managers — the with statement	13
Part 2 — Python's Object Model	14
2.1 Everything is an object.....	14
2.2 Defining your own classes	14
2.3 Dunder methods — the special protocol	14
2.4 Dataclasses — classes without the boilerplate	15
2.5 Inheritance and composition	15
2.6 Duck typing.....	16
2.7 Decorators.....	16
Part 3 — Type Hints and Modern Python	18
3.1 Why type hints exist.....	18
3.2 Basic syntax	18
3.3 Optional, Union, and the syntax.....	18
3.4 Common type names from the typing module.....	18
3.5 Type checking with mypy and pyright	19
3.6 Pydantic — TypeScript-grade validation	19
3.7 Protocols — duck typing with safety	20

3.8 What to watch for in AI-written Python types.....	20
Part 4 — The Python Ecosystem	21
4.1 Python versions	21
4.2 Virtual environments	21
4.3 pip — installing packages	22
4.4 The modern toolchain: uv, ruff, pyright.....	22
4.5 Project files: pyproject.toml	22
4.6 Common ecosystem tools you'll see named	23
Part 5 — Standard Library Heavy Hitters	24
5.1 pathlib — modern file paths	24
5.2 datetime — dates and times	24
5.3 json — reading and writing JSON.....	24
5.4 collections — useful data structures	25
5.5 itertools — iterator algebra	25
5.6 logging — instead of print	25
5.7 subprocess — running other commands.....	26
5.8 os and os.environ	26
Part 6 — Async Python and the GIL.....	27
6.1 The GIL — Python's concurrency reality.....	27
6.2 Three flavors of concurrency in Python.....	27
6.3 asyncio — the modern shape	27
6.4 When to reach for async.....	28
6.5 Common async traps.....	28
Part 7 — Backend Development	29
7.1 The three main frameworks	29
7.2 FastAPI in five minutes	29
7.3 Auto-generated OpenAPI docs	30
7.4 Dependency injection	30
7.5 Databases and ORMs	31
SQLAlchemy basics.....	31
7.6 Authentication in Python	32
7.7 Background tasks and queues.....	32
7.8 Django in brief	33
7.9 Flask in brief	33
7.10 A realistic FastAPI stack for new projects	34
Part 8 — Data Work	35

8.1 numpy — the foundation.....	35
8.2 pandas — tabular data	35
8.3 Pandas anti-patterns	36
8.4 Beyond pandas: polars and duckdb.....	36
8.5 Notebooks: Jupyter and modern alternatives	37
8.6 Visualization	37
8.7 Data validation and cleanup.....	38
Part 9 — AI, ML, and LLM Work	39
9.1 The three layers of AI work	39
9.2 scikit-learn — classical ML in one library	39
9.3 PyTorch — the deep learning standard	39
9.4 Hugging Face — the model hub	40
9.5 Calling LLMs as a service	40
9.6 Streaming responses.....	41
9.7 Structured outputs	41
9.8 Embeddings and vector search	42
9.9 RAG — Retrieval-Augmented Generation	42
9.10 LangChain, LlamaIndex, and the framework question	43
9.11 Agents and tool use	44
9.12 Evaluating LLM features	44
9.13 Watch-outs for AI-written AI code	45
Part 10 — Testing	46
10.1 pytest — the standard.....	46
10.2 Fixtures — setup and teardown	46
10.3 Parametrize — one test, many cases	46
10.4 Mocking	46
10.5 Testing FastAPI	47
10.6 Coverage.....	47
10.7 Python-specific test ideas worth requesting	47
Part 11 — Production Concerns.....	48
11.1 Project layout.....	48
11.2 Configuration	48
11.3 Logging in production	49
11.4 Packaging Python for deployment	49
11.5 ASGI servers	49
11.6 Performance gotchas.....	49

Part 12 — Working with AI Agents in Python	51
12.1 What agents do well in Python.....	51
12.2 What agents struggle with in Python.....	51
12.3 Setting up a Python project for AI agents.....	51
12.4 The Python review checklist	52
12.5 A good Python prompt.....	52
12.6 The verification loop, Python edition	52
12.7 The single best Python habit.....	53
Glossary.....	54
Appendix — Command Cheat Sheet	58
Project setup with uv (recommended).....	58
Project setup with pip + venv (traditional)	58
Daily workflow	58
FastAPI dev.....	58
Useful one-liners	59
Closing Thought.....	60

Introduction

Who this is for

You want to build software. You're not a hands-on developer. You may already have the React & TypeScript Fundamentals doc — this one is its companion, covering Python: the language behind most AI and machine learning, much of modern data work, and a substantial slice of backend development.

Same goal as before: build the mental model so you can review AI-generated code with confidence, recognize good patterns from bad, and direct your agent rather than be directed by it.

How this relates to the React & TypeScript doc

Cross-cutting concepts — HTTP, REST, databases, authentication, deployment, CI/CD, monorepos, working with AI agents — were covered in detail in the React & TypeScript doc. This document references them but does not repeat them. When something appears as "see the React & TypeScript doc, Part X," go read that section if you haven't already.

What this doc covers exclusively:

- Python the language: syntax, idioms, what makes it different from TypeScript.
- The Python ecosystem: pip, virtual environments, packaging.
- Python's object model: classes, dataclasses, duck typing, magic methods.
- Type hints and Pydantic: bringing TypeScript-like safety to Python.
- Async Python and the GIL.
- Backend development with FastAPI, Django, and Flask.
- Data work with numpy, pandas, and the modern alternatives.
- AI/ML/LLM work: PyTorch, scikit-learn, LangChain, the OpenAI and Anthropic SDKs, RAG.
- Testing with pytest.
- Python-specific anti-patterns and the failures AI agents make in Python.

Why Python matters in the AI era

Python is the lingua franca of AI and machine learning. Every major model is trained, fine-tuned, and served from Python. Every major library — PyTorch, TensorFlow, scikit-learn, LangChain, LlamaIndex — has Python as its primary surface. Most LLM tutorials, agents, and tools assume Python.

Python is also one of the most popular backend languages. FastAPI, Django, and Flask power large parts of the internet. Data engineering teams live in Python. Scripting and automation lean on Python.

In the AI-native era, you'll meet Python whether or not you set out to. This document makes it readable.

Key shift from JavaScript/TypeScript

Python is a different mental model. Significant whitespace replaces curly braces. Everything is an object. There's no event loop by default. The package ecosystem looks different. Async exists but is opt-in, not pervasive. Once you internalize these differences, the rest follows.

How to use this document

Same as the React doc. Read once linearly to build the mental model, then keep it open as reference. Look for three kinds of insets throughout:

- **Callouts** — the key idea or shift in thinking for a section.
- **Anti-patterns** — specific Python mistakes you'll encounter and want to recognize.
- **Practical tips** — direct advice from real projects.

Part 1 — The Python Language Essentials

1.1 The shape of Python

Python's syntax is famously clean. There are no curly braces and no semicolons; indentation defines blocks of code. A four-space indent is the convention, and it is enforced — code that's indented inconsistently is a syntax error.

```
def greet(name):  
    if name:  
        print(f"Hello, {name}")  
    else:  
        print("Hello, stranger")
```

⚠ Anti-pattern

Mixing tabs and spaces for indentation. Python 3 forbids it. Modern editors are configured to insert spaces; if you copy code in from somewhere else, normalize the whitespace first. Most lint tools (ruff, black) will fix this automatically.

1.2 Variables and basic types

Variables are created by assignment. No declaration keyword, no type annotation required.

```
name = "Maya"           # str  
age = 32                 # int  
height = 1.68           # float  
is_active = True        # bool (capital T/F)  
nothing = None          # None – Python's null  
  
tags = ["a", "b", "c"]  # list  
coords = (10, 20)       # tuple – immutable  
user = {"name": "Maya", "age": 32} # dict  
unique = {1, 2, 3}      # set
```

Notice: bool is capitalized, None replaces null, and dict literals use curly braces (different from JavaScript objects but visually similar).

1.3 Strings and f-strings

Python strings are immutable. The modern way to format them is f-strings — prefix the string with f and embed expressions in curly braces.

```
name = "Maya"  
greeting = f"Hello, {name}!"      # "Hello, Maya!"  
math = f"2 + 2 = {2 + 2}"        # "2 + 2 = 4"  
formatted = f"Price: ${price:.2f}" # "Price: $19.99"
```


⚠ Anti-pattern

Older code uses `str.format()` or `%-formatting`: `"Hello, %s" % name` or `"Hello, {}".format(name)`. They still work but are noisier and slower. Use f-strings in any code written today.

1.4 Lists, tuples, dicts, sets

Python's four core collection types each have a sweet spot:

Type	When to use	Mutable?
list	Ordered sequence you'll modify	Yes
tuple	Ordered sequence that won't change (coordinates, fixed records)	No
dict	Key → value mapping; look up by name	Yes
set	Unique items; fast membership tests	Yes

```
# List operations
tags = ["react", "python"]
tags.append("sql")           # ["react", "python", "sql"]
tags[0]                      # "react"
tags[-1]                    # "sql" - negative index = from end
tags[1:3]                   # ["python", "sql"] - slicing
len(tags)                   # 3

# Dict operations
user = {"name": "Maya", "age": 32}
user["name"]                 # "Maya"
user.get("email", "unknown") # "unknown" - safe default
"name" in user               # True
```

✓ Practical tip

Use `dict.get(key, default)` instead of `dict[key]` when the key may not exist. `KeyError` is the most common Python exception caused by assuming a key is there. The `.get()` form is safe and reads cleanly.

1.5 Slicing

Python's slice syntax works on any sequence — lists, tuples, strings. It's one of Python's nicest features.

```
items = [0, 1, 2, 3, 4, 5]
items[2:5]      # [2, 3, 4] - from index 2 up to (not including) 5
items[:3]       # [0, 1, 2] - from start
```

```
items[3:]      # [3, 4, 5]    – to end
items[::2]     # [0, 2, 4]    – every 2nd
items[::-1]    # [5, 4, 3, 2, 1, 0] – reversed

text = "Hello"
text[1:4]      # "ell"
```

1.6 Control flow

```
# if / elif / else
if age >= 18:
    print("adult")
elif age >= 13:
    print("teen")
else:
    print("child")

# for loops – iterate over things, not indexes
for tag in tags:
    print(tag)

# enumerate when you need the index too
for index, tag in enumerate(tags):
    print(index, tag)

# while loops
while not done:
    do_work()

# match (Python 3.10+) – structural pattern matching
match status:
    case "loading": show_spinner()
    case "error":   show_error()
    case _:         show_content()    # _ is default
```

⚠ Anti-pattern

Using `range(len(items))` just to loop over a list: `for i in range(len(items)): print(items[i])`. Use `for item in items:` directly. If you need the index, use `enumerate(items)`. The `range(len())` form is a strong signal someone learned a different language first.

1.7 Comprehensions

Python's signature feature. A comprehension builds a new collection from another in one line.

```
# List comprehension
squares = [n * n for n in range(10)]

# With a filter
evens = [n for n in range(20) if n % 2 == 0]
```

```
# Dict comprehension
lookup = {user.id: user for user in users}

# Set comprehension
unique_tags = {tag.lower() for post in posts for tag in post.tags}
```

✓ Practical tip

List comprehensions are idiomatic Python. They're the preferred way to transform a collection — clearer than map/filter for most readers. But keep them shallow: nesting more than one for clause hurts readability fast. If the comprehension would span multiple lines or carry complex logic, use a regular for loop instead.

1.8 Functions

```
def greet(name, excited=False):
    suffix = "!" if excited else "."
    return f"Hello, {name}{suffix}"

greet("Maya")           # "Hello, Maya."
greet("Maya", excited=True) # "Hello, Maya!"
greet(name="Maya", excited=True) # keyword arguments – explicit
```

Default values, keyword arguments, and the * and ** operators give Python flexible function signatures.

```
# *args collects positional args into a tuple
def total(*numbers):
    return sum(numbers)

total(1, 2, 3)          # 6

# **kwargs collects keyword args into a dict
def configure(**options):
    print(options)

configure(theme="dark", lang="en") # {'theme': 'dark', 'lang': 'en'}
```

Mutable default arguments — the classic trap

⚠ Anti-pattern

`def add_item(item, items=[]): items.append(item); return items` — the list is created once at function definition and shared across all calls. Successive calls accumulate state. Always use `None` as the default and create the list inside: `def add_item(item, items=None): if items is None: items = []`. This trap catches everyone exactly once.

1.9 Iteration and the iterator protocol

Many Python features rely on iterators — objects that can be looped over. The standard tools you'll see constantly:

```
# enumerate – get (index, value) pairs
for i, name in enumerate(names):
    print(i, name)

# zip – iterate over multiple sequences in parallel
for name, age in zip(names, ages):
    print(name, age)

# range – generate integers
for i in range(10):          # 0..9
    print(i)
for i in range(5, 15, 2):    # start, stop, step
    print(i)
```

1.10 Generators — lazy sequences

A function with `yield` produces a generator — a lazy iterator that produces values one at a time, on demand, without holding them all in memory.

```
def line_lengths(filename):
    with open(filename) as f:
        for line in f:                # files are themselves iterators
            yield len(line)

# Generators are consumed lazily – perfect for huge files
for length in line_lengths("huge.log"):
    process(length)
```

✓ Practical tip

Use generators for large or potentially infinite sequences. They process data as it flows through instead of loading the whole thing into memory. If you find yourself building a list and then iterating through it once, a generator is usually a better shape.

1.11 Exceptions

```
try:
    data = json.loads(text)
except json.JSONDecodeError as e:
    print(f"Bad JSON: {e}")
    data = {}
except (KeyError, ValueError) as e:    # catch multiple types
    print(f"Bad data: {e}")
finally:
    cleanup()                          # runs whether or not there was an error

# Raising your own
if amount < 0:
```

```
raise ValueError("amount must be non-negative")
```

⚠ Anti-pattern

Bare except: with no exception type. This catches every exception including KeyboardInterrupt and SystemExit, hiding bugs and making programs impossible to stop with Ctrl+C. Always specify the exception type, or use except Exception: at minimum. Bare except: should never appear in production code.

⚠ Anti-pattern

except: pass — swallowing errors silently. Either log them, re-raise them, handle them visibly, or don't catch them at all. Silent failures are how bugs hide.

1.12 Context managers — the with statement

Python's with statement guarantees cleanup, even if an error occurs. The classic use is files.

```
# WRONG — file might not close if there's an error
f = open("data.txt")
data = f.read()
f.close()

# RIGHT — file always closes
with open("data.txt") as f:
    data = f.read()
# file is closed here, automatically
```

Many resources support the with statement: file handles, database connections, network sockets, locks, even temporary directories.

✓ Practical tip

If a library returns something that needs cleanup — a connection, a file, a session — check whether it supports with. If it does, use it. Forgetting to close resources is a classic Python bug AI agents make, especially in long-running scripts.

Part 2 — Python's Object Model

2.1 Everything is an object

In Python, everything is an object — including numbers, strings, functions, classes, and modules. Every object has a type (which determines what you can do with it) and an identity (a unique address in memory). You don't need to think about this constantly, but it explains a lot of Python's behavior.

```
type(42)           # <class 'int'>
type("hello")     # <class 'str'>
type(print)       # <class 'builtin_function_or_method'>
type([1, 2, 3])   # <class 'list'>
```

2.2 Defining your own classes

```
class User:
    def __init__(self, name, age):    # constructor
        self.name = name
        self.age = age

    def greet(self):                  # method
        return f"Hello, I'm {self.name}"

maya = User("Maya", 32)
maya.name      # "Maya"
maya.greet()   # "Hello, I'm Maya"
```

Note: methods take `self` as their first parameter — Python is explicit about this. `self` is the instance the method is being called on. You don't pass it when calling — Python fills it in automatically.

2.3 Dunder methods — the special protocol

Methods starting and ending with double underscores ("dunders") give your class special behaviors. `__init__` is the constructor; many others customize how your object behaves with Python's built-in syntax.

Dunder	Lets you do	Example
<code>__init__</code>	Construct the object	<code>User("Maya")</code>
<code>__repr__</code>	Control its string for debugging	<code>print(repr(user))</code>
<code>__str__</code>	Control its display string	<code>str(user)</code> or <code>print(user)</code>
<code>__eq__</code>	Define equality (<code>==</code>)	<code>user1 == user2</code>
<code>__hash__</code>	Use as a dict key or in a set	<code>{user: balance}</code>
<code>__len__</code>	Use with <code>len()</code>	<code>len(my_collection)</code>

Dunder	Lets you do	Example
<code>__iter__</code>	Loop with for	for x in my_obj:
<code>__getitem__</code>	Index with []	my_obj[key]
<code>__enter__</code> / <code>__exit__</code>	Use as a context manager	with my_obj as x:

Why this matters

When AI generates a class, scan the dunder. A class that supports `__repr__`, `__eq__`, and `__hash__` is more usable in the rest of your code. Missing them means debugging output is unhelpful and the object can't be used in collections.

2.4 Dataclasses — classes without the boilerplate

Most classes are just bundles of data. The `@dataclass` decorator generates `__init__`, `__repr__`, and `__eq__` for you from type-annotated fields.

```
from dataclasses import dataclass

@dataclass
class User:
    name: str
    age: int
    is_admin: bool = False

maya = User(name="Maya", age=32)
print(maya)          # User(name='Maya', age=32, is_admin=False) - auto __repr__
```

✓ Practical tip

Use `@dataclass` for any class that's primarily a data container. It saves writing `__init__` and `__repr__` by hand, and the type annotations on fields double as documentation. For data that comes from outside (API responses, config files, form input), prefer Pydantic models (Part 3) — they add runtime validation.

2.5 Inheritance and composition

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        raise NotImplementedError

class Dog(Animal):
    def speak(self):
        return "Woof"

class Cat(Animal):
```

```
def speak(self):  
    return "Meow"
```

Python supports inheritance — even multiple inheritance — but the modern style favors composition over deep inheritance. Inherit when there's a true "is-a" relationship; compose when one object has another.

⚠ Anti-pattern

Deep inheritance hierarchies. A class that inherits from a class that inherits from another class that inherits from a base is a smell. By the time you're reading the third level, you've lost track of what attributes and methods are available. Flatten with composition or use protocols (Part 3).

2.6 Duck typing

"If it walks like a duck and quacks like a duck, it's a duck." Python checks behavior, not type. A function that needs something to iterate over doesn't care whether you pass a list, a tuple, a generator, or a custom object — as long as it can be iterated.

```
def total_lines(thing):  
    return sum(1 for _ in thing)  
  
total_lines(["a", "b", "c"])          # works  
total_lines(("a", "b"))              # works  
total_lines(open("file.txt"))        # works – files are iterators  
total_lines(line for line in stream) # works – generators too
```

Modern alternative: Protocols

Type hints in modern Python let you say "this argument must support these methods" without naming a specific class. See Part 3 for typing.Protocol — the formal way to express duck-typed interfaces.

2.7 Decorators

A decorator is a function that wraps another function to add behavior. They use the @ syntax and you'll see them constantly — for routing in web frameworks, caching, timing, authentication.

```
from functools import lru_cache  
  
@lru_cache(maxsize=100)  
def expensive(n):  
    # ... slow computation ...  
    return result  
  
# Now repeated calls with the same arg are cached.  
expensive(42)    # slow  
expensive(42)    # instant
```


Other decorators you'll meet: `@staticmethod`, `@classmethod`, `@property`, FastAPI's `@app.get("/")`, pytest's `@pytest.fixture`.

Part 3 — Type Hints and Modern Python

3.1 Why type hints exist

Python is dynamically typed: a variable can hold any kind of value, with no checking. Like JavaScript, this is flexible and bug-prone. Unlike TypeScript, Python added types as optional annotations rather than a separate language.

Type hints are checked by external tools (mypy, pyright) — not by Python itself. They're erased at runtime; the program runs exactly the same with or without them. They exist for editors, type checkers, and human readers.

The deal

Type hints cost you a little extra writing. They give you autocomplete, refactor safety, and a precise specification of what each function takes and returns — the same value proposition as TypeScript. For AI-assisted coding, they're doubly valuable: they constrain what the agent can do and give you a precise way to express what you want.

3.2 Basic syntax

```
name: str = "Maya"
age: int = 32
scores: list[int] = [90, 85, 88]
user: dict[str, int] = {"age": 32}

def greet(name: str, excited: bool = False) -> str:
    suffix = "!" if excited else "."
    return f"Hello, {name}{suffix}"
```

Read the function signature as: "greet takes a str name and an optional bool excited, and returns a str."

3.3 Optional, Union, and the | syntax

```
# Python 3.10+
name: str | None = None          # might be a string or None
id: int | str = "abc"            # int or str

# Older syntax (still common in code today)
from typing import Optional, Union
name: Optional[str] = None
id: Union[int, str] = "abc"
```

3.4 Common type names from the typing module

Annotation	Meaning	
list[T]	List of T	

Annotation	Meaning	
dict[K, V]	Dict with K keys and V values	
tuple[T, ...]	Tuple of any length, all T	
set[T]	Set of T	
T None	T or None (Optional[T])	
Callable[[A, B], R]	Function taking A, B and returning R	
Any	Disables type checking — avoid	
Literal["a", "b"]	Only those exact values allowed	

⚠ Anti-pattern

Annotating everything as Any. It disables checking and propagates. Used genuinely ("this can truly be anything"), it's fine; used to silence the type checker, it's a debt. When AI writes Any, ask for the real type.

3.5 Type checking with mypy and pyright

Python doesn't check types at runtime. You need a checker:

Tool	Notes	
mypy	The original. Mature, configurable, slower.	
pyright	From Microsoft. Faster, smarter inference, powers VS Code's Pylance.	
ty / pyrefly	Newer fast checkers in Rust	

In CI, run `pyright src/` or `mypy src/` to catch type errors before they hit production.

3.6 Pydantic — TypeScript-grade validation

Type hints alone don't validate data at runtime. Pydantic adds runtime validation on top of type hints — it's become near-universal in modern Python, especially for APIs and AI work.

```
from pydantic import BaseModel, EmailStr

class User(BaseModel):
    id: int
    name: str
    email: EmailStr          # validated email format
    is_admin: bool = False
```

```
# From a dict - validates and converts
user = User(**raw_json)          # raises ValidationError if bad

# Back to a dict / JSON
user.model_dump()                # plain dict
user.model_dump_json()           # JSON string
```

Pydantic is to Python what Zod is to TypeScript

Validate untrusted data at the boundary — API requests, file inputs, LLM outputs. The type hints become real, enforced guarantees. FastAPI uses Pydantic internally, which is a big part of why it has so much momentum.

3.7 Protocols — duck typing with safety

`typing.Protocol` lets you express "anything with these methods," so duck typing remains flexible but the type checker can verify it.

```
from typing import Protocol

class Sized(Protocol):
    def __len__(self) -> int: ...

def first_half(thing: Sized):    # anything with __len__ works
    return thing[:len(thing) // 2]
```

3.8 What to watch for in AI-written Python types

- Excessive `Any` — same smell as in TypeScript.
- Missing return type annotations on non-trivial functions — they're the most useful annotation.
- `dict[str, Any]` when a Pydantic model or dataclass would be clearer.
- `# type: ignore` comments — every one is a debt.
- Inconsistent style — some files heavily annotated, others not. Adopt a project standard in `CLAUDE.md`.

Part 4 — The Python Ecosystem

4.1 Python versions

There are two Pythons that matter: 3.x and "that one" — Python 2, which was retired in 2020 and which you should never touch in new projects. Within Python 3, the relevant versions are 3.10 through the latest (currently 3.13).

Each minor version (3.10 → 3.11 → 3.12 → 3.13) adds features and improves performance. Pin a version per project.

✓ Practical tip

Install Python with pyenv (mac/linux) or via the python.org installer (Windows), not via your system package manager. System Python is for the OS — touching it can break your computer. Project Python lives in pyenv or asdf, and gets isolated further with virtual environments.

4.2 Virtual environments

Python's default behavior is to install packages globally — into your one Python installation. That's a disaster. Different projects need different versions of the same library; you cannot have both at once globally.

A virtual environment ("venv") is a folder that contains a private Python and a private set of installed packages. You create one per project.

```
# Create one
python -m venv .venv

# Activate it
source .venv/bin/activate      # macOS / Linux
.venv\Scripts\activate        # Windows

# Your shell prompt now shows (.venv)
# pip installs go into .venv, not your system Python

# Deactivate when done
deactivate
```

⚠ Anti-pattern

Running pip install without an active virtual environment. Packages go into your system Python or user Python, conflict with other projects, and eventually break something. Always venv first.

⚠ Anti-pattern

`sudo pip install <anything>`. This installs system-wide and modifies files the OS owns. Combined with package version conflicts, it's how machines get into unrecoverable states. There is essentially never a good reason to use `sudo` with `pip`.

4.3 pip — installing packages

```
# Inside an activated venv:
pip install fastapi           # latest version
pip install "fastapi>=0.110" # version constraint
pip install -r requirements.txt # batch install from a file
pip install -e .              # install current project in editable mode
pip list                      # what's installed
pip freeze > requirements.txt # snapshot current versions
```

4.4 The modern toolchain: uv, ruff, pyright

In the last couple of years, a new generation of Rust-based Python tools has replaced the old stack. They're 10-100x faster and unify multiple older tools.

Modern	Replaces	Job
uv	pip, pip-tools, venv, pyenv, virtualenv	Package install, venv mgmt, version mgmt
ruff	flake8, black, isort, pylint, pyupgrade	Lint + format
pyright / ty	(complements mypy)	Type check
mise / asdf	pyenv	Version manager (cross-language)

✓ Practical tip

For new projects in 2026, default to `uv` for everything package-related and `ruff` for lint+format. `uv install` replaces `pip install`, `uv run` replaces `python`, and it's so fast that what used to be a 30-second `pip install` now takes a second or two.

4.5 Project files: pyproject.toml

The modern home for project metadata, dependencies, and tool configuration. Replaces older `setup.py` and `requirements.txt` patterns.

```
# pyproject.toml – a typical snippet
[project]
name = "my-app"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
```

```
"fastapi">=0.110",
"pydantic">=2.5",
"sqlalchemy">=2.0",
]

[project.optional-dependencies]
dev = ["pytest", "ruff", "pyright"]

[tool.ruff]
line-length = 100

[tool.pytest.ini_options]
testpaths = ["tests"]
```

4.6 Common ecosystem tools you'll see named

Tool	Job	Status
pip	Package installer	Standard
uv	All-in-one package + venv manager	Modern default
poetry	Project + dependency manager	Mature, still common
pdm	Project manager	Alternative to poetry
pipx	Install Python apps system-wide in isolation	Useful
conda / mamba	Environment + package manager	Common in data science / ML
ruff	Linter + formatter	Modern default
black	Formatter	Older default; ruff format replaces it
mypy / pyright	Type checker	Standard
pre-commit	Run checks before git commit	Standard

conda vs pip

Conda (and the faster mamba) is a separate package manager popular in data science and ML because it can install non-Python binaries (CUDA, MKL, native libraries). For pure Python web/backend work, use pip or uv. For heavy ML environments, conda often pays off.

Part 5 — Standard Library Heavy Hitters

Python's standard library is enormous and well-maintained. Knowing what's in it prevents you from installing a third-party package for something that already ships with Python. AI agents sometimes reach for a package when a standard-library tool exists — push back when you can.

5.1 pathlib — modern file paths

```
from pathlib import Path

p = Path("data") / "users.json"      # construct paths with /
p.exists()                          # bool
p.is_file()                          # bool
p.read_text()                        # full contents as str
p.write_text("hello")                # write a string
p.suffix                             # ".json"
p.parent                             # Path("data")

for f in Path("logs").glob("*.txt"):  # iterate matching files
    print(f)
```

⚠ Anti-pattern

Using `os.path` string concatenation: `os.path.join(os.path.dirname(__file__), "..", "data", "users.json")`. `pathlib` is shorter, safer, and reads like English. Use it for all new code.

5.2 datetime — dates and times

```
from datetime import datetime, timedelta, timezone

now = datetime.now(timezone.utc)      # always specify tz!
future = now + timedelta(days=7)
iso = now.isoformat()                 # "2026-05-31T12:34:56+00:00"
parsed = datetime.fromisoformat(iso)
```

⚠ Anti-pattern

`datetime.now()` without a timezone, returning a "naive" datetime that the rest of your system has to guess about. Always pass `timezone.utc`. Naive datetimes are responsible for an absurd amount of production wreckage — DST bugs, off-by-an-hour errors, daylight saving disasters.

5.3 json — reading and writing JSON

```
import json

data = json.loads(text)               # str → dict
text = json.dumps(data, indent=2)     # dict → str
```



```
with open("data.json") as f:
    data = json.load(f)                # read from file

with open("out.json", "w") as f:
    json.dump(data, f, indent=2)       # write to file
```

5.4 collections — useful data structures

```
from collections import Counter, defaultdict, deque

# Counter – tally things
c = Counter(["a", "b", "a", "c", "a"])
c.most_common(2)                       # [('a', 3), ('b', 1)]

# defaultdict – auto-create missing keys
groups = defaultdict(list)
for item in items:
    groups[item.category].append(item)

# deque – fast push/pop at both ends
from collections import deque
q = deque([1, 2, 3])
q.appendleft(0)                         # [0, 1, 2, 3]
q.pop()                                # 3
```

5.5 itertools — iterator algebra

A set of building blocks for working with iterators. The handful you'll meet often:

Function	Does	Example
chain	Stitch iterables together	chain([1,2], [3,4]) → 1,2,3,4
islice	Take a slice from an iterator	islice(stream, 10)
groupby	Group consecutive equal items	for k, group in groupby(items):
combinations	All combinations	combinations([1,2,3], 2) → (1,2), (1,3), (2,3)
product	Cartesian product	product([1,2], [3,4]) → (1,3), (1,4), (2,3), (2,4)

5.6 logging — instead of print

```
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

logger.debug("detailed info")
logger.info("normal events")
```

```
logger.warning("something looks off")
logger.error("something failed", exc_info=True)    # include traceback
```

⚠ Anti-pattern

`print()` everywhere in production code. `print()` is for ad-hoc debugging. Real applications use logging, because it has levels (debug/info/warning/error), can be filtered by environment, sent to files or aggregators (Datadog, Sentry), and turned off in production for noisy debug lines without code changes. Configure logging once, then use `logger.info()` in place of `print()`.

5.7 subprocess — running other commands

```
import subprocess

result = subprocess.run(
    ["git", "status", "--short"],
    capture_output=True, text=True, check=True
)
print(result.stdout)
```

⚠ Anti-pattern

`subprocess.run("git status", shell=True)` — passing a string with `shell=True` invites shell-injection vulnerabilities if any input is user-controlled. Use the list form: `["git", "status"]`. It's also faster and avoids quoting confusion.

5.8 os and os.environ

```
import os

api_key = os.environ["OPENAI_API_KEY"]           # raises if missing
api_key = os.environ.get("OPENAI_API_KEY")       # None if missing
api_key = os.environ.get("API_URL", "localhost") # default
```

✓ Practical tip

Don't put secrets or environment-specific config in source code. Read them from `os.environ`. For local development, use `python-dotenv` to load them from a `.env` file (which is in `.gitignore`). For production, your hosting provider has a secrets manager.

Part 6 — Async Python and the GIL

6.1 The GIL — Python's concurrency reality

CPython (the standard Python) has a Global Interpreter Lock (GIL): only one thread executes Python bytecode at a time. This means:

- Multiple threads do not speed up CPU-bound Python work — they take turns on one core.
- Multiple threads do help with I/O-bound work (waiting on networks, disk, databases) — while one waits, another runs.
- To use multiple cores for Python computation, use multiprocessing (separate processes) or release the GIL with C extensions (numpy does this).

The GIL in 2026

Python 3.13 introduced an experimental free-threaded mode (no GIL). It's not yet the default, and most libraries don't yet support it. For now, assume the GIL exists when reasoning about Python concurrency.

6.2 Three flavors of concurrency in Python

Approach	Best for	Cost
threading	I/O-bound with many waits	GIL means no parallel CPU; sync code awkward
multiprocessing	CPU-bound parallel work	Heavy; data must be picklable to cross processes
asyncio	Many concurrent I/O operations	Code must be written async; mixes poorly with sync

⚠ Anti-pattern

Using threading for CPU-heavy work — number crunching, ML, large data transforms. The GIL makes it no faster than single-threaded. Use multiprocessing (or move the heavy bit to numpy/pandas/native code) for actual parallelism on CPU.

6.3 asyncio — the modern shape

Async Python uses `async def` and `await` — the same syntax as JavaScript. An async function returns a coroutine; you must either await it or run it through an event loop.

```
import asyncio
import httpx

async def fetch_user(client, user_id):
```

```
response = await client.get(f"/users/{user_id}")
return response.json()

async def main():
    async with httpx.AsyncClient(base_url="https://api.example.com") as client:
        # Fetch many users concurrently
        tasks = [fetch_user(client, i) for i in range(1, 11)]
        users = await asyncio.gather(*tasks)
        return users

users = asyncio.run(main())
```

Mental model

async/await lets one thread juggle many in-flight I/O operations. When a coroutine awaits, the event loop runs another ready coroutine. Perfect for APIs, web scraping, and database-heavy work. Useless for pure number crunching.

6.4 When to reach for async

- FastAPI endpoints — they're async by default and benefit enormously.
- Code that calls many APIs in sequence — make them concurrent with `gather()`.
- Bots, scrapers, real-time pipelines.
- Servers handling many simultaneous clients.

And when not to:

- Simple scripts. The complexity isn't worth it.
- CPU-heavy code. async won't help; you need processes or native libraries.
- Code that needs to call lots of sync libraries. Mixing sync and async is awkward.

6.5 Common async traps

⚠ Anti-pattern

Forgetting `await`. `result = fetch_user(client, 1)` gives you a coroutine object, not the result. The function never actually runs. AI agents miss this surprisingly often when porting sync code to async. If a variable contains "<coroutine object ...>" instead of data, look for a missing `await`.

⚠ Anti-pattern

Calling a sync blocking function inside async code. `requests.get()` or `time.sleep()` inside an async function blocks the entire event loop — undoing the whole point of async. Use `httpx.AsyncClient` instead of `requests`, `asyncio.sleep()` instead of `time.sleep()`, and async database libraries (`asyncpg`, `motor`) instead of sync ones.

Part 7 — Backend Development

This part covers the Python side of a typical web backend. For the React/TS frontend that consumes it, and for cross-cutting concerns like HTTP, REST, deployment, and CI/CD, see the React & TypeScript Fundamentals doc, Parts 6 and 9–10.

7.1 The three main frameworks

Framework	Style	Best for
FastAPI	Modern, async-first, Pydantic-native, auto-docs	New APIs and modern services — recommended default
Django	Batteries-included; ORM, admin, auth, templates	Content-heavy apps, admin-heavy products, traditional web
Flask	Minimal, flexible, sync-first	Small services, prototypes, when you want to assemble pieces

Default choice in 2026

For a new Python API to pair with a React frontend, use FastAPI. It's async, generates OpenAPI docs automatically, uses Pydantic for input/output validation, and has the best developer experience. Django remains excellent for content/admin-heavy apps; Flask is the old reliable for small services.

7.2 FastAPI in five minutes

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class UserCreate(BaseModel):
    name: str
    email: str
    age: int

class User(BaseModel):
    id: int
    name: str
    email: str
    age: int

@app.get("/users/{user_id}")
async def get_user(user_id: int) -> User:
    user = db.fetch_user(user_id)
    if user is None:
        raise HTTPException(status_code=404, detail="Not found")
    return user
```

```
@app.post("/users", status_code=201)
async def create_user(payload: UserCreate) -> User:
    return db.create_user(payload)
```

Things to notice:

- Endpoints are async functions decorated with `@app.get(...)` or `@app.post(...)`.
- URL parameters are typed function arguments (`user_id: int`) — FastAPI converts and validates automatically.
- Request bodies are Pydantic models — FastAPI parses, validates, and produces helpful errors automatically.
- Return type annotations both document and validate the response.

7.3 Auto-generated OpenAPI docs

FastAPI generates a full OpenAPI (Swagger) spec from your routes and Pydantic models — visit `/docs` in dev to see interactive documentation, or `/openapi.json` for the raw spec. Frontends can then generate fully typed clients from that spec (see the React doc, Part 10.12).

✓ Practical tip

End-to-end type safety from Python to TypeScript: define your models in Pydantic, run a tool like `openapi-typescript` on the generated spec, and your frontend gets fully typed API calls automatically. Refactor a Python model, recompile the frontend, broken call sites light up red. This is one of the highest-leverage setups available for AI-assisted full-stack work.

7.4 Dependency injection

FastAPI has a clean dependency-injection system — declare what each endpoint needs, FastAPI provides it.

```
from fastapi import Depends

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/users/{user_id}")
async def get_user(user_id: int, db: Session = Depends(get_db)) -> User:
    return db.get_user(user_id)
```

Common dependencies: database sessions, current user from auth token, API clients, cached config. The same machinery is used for authentication, rate limiting, and feature flags.

7.5 Databases and ORMs

Python's main database libraries and ORMs:

Library	Style	Notes
SQLAlchemy	Mature ORM + query builder	The standard. Heavy but powerful.
SQLModel	Pydantic-based ORM on top of SQLAlchemy	From FastAPI's author; less mature
Tortoise / Piccolo	Async-native ORMs	Newer, smaller communities
Django ORM	Comes with Django	Excellent — only available inside Django
asyncpg	Direct PostgreSQL driver	Use under an ORM, or directly for max speed
psycopg2 / psycopg3	Sync PostgreSQL driver	psycopg3 is current
Alembic	Schema migrations for SQLAlchemy	Standard

On databases

Default to PostgreSQL. The reasoning from the React doc Part 10.4 applies identically: mature, free, supports JSON columns, has a vector extension (pgvector) for AI work, supported everywhere. SQLite is great for development and small apps. Redis for caching and queues. Other databases only when the project has a specific reason.

SQLAlchemy basics

```
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str]
    email: Mapped[str] = mapped_column(unique=True)

# Query
user = session.execute(select(User).where(User.id == 1)).scalar_one()

# Insert
session.add(User(name="Maya", email="maya@example.com"))
session.commit()
```

⚠ Anti-pattern

Constructing SQL queries with f-strings or string concatenation: `f"SELECT * FROM users WHERE id = {user_id}"`. This is SQL injection. Always pass values as parameters: `text("SELECT * FROM users WHERE id = :id")`, `{"id": user_id}`. ORMs do this for you automatically — another good reason to use one.

7.6 Authentication in Python

Building blocks specific to Python (auth concepts overall are covered in the React doc, Part 10.6):

Library	Purpose	
passlib / bcrypt / argon2-cffi	Hash passwords	
python-jose / PyJWT	Encode/decode JWT tokens	
authlib	OAuth client/server	
fastapi-users	Plug-in auth for FastAPI	
Django auth	Comes built into Django	
Authlib + httpx-oauth	Social logins for FastAPI	

✓ Practical tip

Same advice as the React doc: don't roll your own auth from scratch. For FastAPI, `fastapi-users` or a hosted service (Clerk, Auth0, Supabase Auth) handles flows and edge cases you don't want to debug at 2am. Django's built-in auth is good for Django projects.

7.7 Background tasks and queues

Some work doesn't fit in a request — sending emails, generating reports, running ML inference, scheduled jobs. Python has several options:

Tool	Style	Notes
Celery	Distributed task queue with Redis/RabbitMQ broker	Industry standard, mature, heavy
RQ	Simpler Redis-based queue	Good for smaller projects
Dramatiq	Modern alternative to Celery	Cleaner API
FastAPI BackgroundTasks	Fire-and-forget in the same process	For very light work

Tool	Style	Notes
APScheduler	In-process cron	Scheduled jobs without a separate worker
Temporal / Inngest / Trigger.dev	Modern workflow engines	Durable, retryable, observable

⚠ Anti-pattern

Doing slow work inside the HTTP request. "Just put it in the handler" is fast to ship and a nightmare to operate — timeouts, retries become impossible, and your API gets blocked. If the operation can take more than ~1 second and isn't required to return data, push it to a queue.

7.8 Django in brief

Django is a full-stack web framework: ORM, admin UI, authentication, templates, forms, migrations — all included. The cost is opinionation: you do things the Django way. Excellent choice for content-heavy apps, e-commerce, internal tools, and anything that benefits from the free admin UI.

```
# models.py
from django.db import models

class User(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)

# views.py
from django.shortcuts import get_object_or_404
from django.http import JsonResponse

def user_detail(request, user_id):
    user = get_object_or_404(User, pk=user_id)
    return JsonResponse({"name": user.name, "email": user.email})
```

Django REST Framework (DRF) adds REST-API capabilities on top — common combo for content APIs. Django Ninja brings FastAPI-like syntax to Django.

7.9 Flask in brief

Flask is minimal. It gives you routing and templating; everything else (database, auth, forms) you assemble from libraries. Excellent for small services where Django would be overkill and FastAPI's async is unnecessary.

```
from flask import Flask, jsonify

app = Flask(__name__)
```

```
@app.get("/users/<int:user_id>")
def get_user(user_id):
    user = db.fetch_user(user_id)
    return jsonify(user.to_dict())
```

7.10 A realistic FastAPI stack for new projects

Mirroring the React doc's "realistic stack" — for the Python backend half:

- **Framework:** FastAPI.
- **Database:** PostgreSQL (hosted on Supabase, Neon, Railway, or RDS).
- **ORM:** SQLAlchemy 2.0 + Alembic for migrations. SQLModel as a lighter alternative.
- **Validation:** Pydantic v2 — already built into FastAPI.
- **Async HTTP client:** httpx (use AsyncClient).
- **Auth:** fastapi-users, or a hosted provider (Clerk, Auth0).
- **Background jobs:** Celery + Redis (mature), or Dramatiq.
- **Caching / queues:** Redis.
- **Testing:** pytest + httpx test client.
- **Type checking:** pyright in strict mode.
- **Lint/format:** ruff.
- **Package manager:** uv.
- **Hosting:** Render, Railway, Fly.io for containers; Vercel Functions / AWS Lambda for serverless.
- **Logging / errors:** structlog + Sentry.

Part 8 — Data Work

Python's data ecosystem is a quiet superpower. Even if you're not a data scientist, you'll meet pandas and numpy when working with CSVs, processing API responses, or preparing data for ML.

8.1 numpy — the foundation

numpy gives you fast multi-dimensional arrays and the math operations to manipulate them. Almost every other data and ML library is built on it.

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
a.shape      # (5,)
a * 2        # array([2, 4, 6, 8, 10]) - element-wise!
a + a        # array([2, 4, 6, 8, 10])
a.mean()     # 3.0
a.sum()      # 15

# 2D arrays (matrices)
m = np.array([[1, 2], [3, 4]])
m.shape      # (2, 2)
m.T          # transpose
```

Key idea: vectorization

Operations on numpy arrays act on every element at once, in highly optimized C code. A loop like `for x in arr: result.append(x * 2)` is dramatically slower than `arr * 2`. The single biggest performance lesson in Python data work is: don't loop, vectorize.

8.2 pandas — tabular data

pandas adds DataFrame and Series — labeled, typed tables. It's the de facto tool for exploring and transforming structured data in Python.

```
import pandas as pd

df = pd.read_csv("sales.csv")
df.head()                # first 5 rows
df.describe()            # summary stats per column
df.dtypes                # column types
df.shape                 # (rows, cols)

# Selecting
df["price"]               # one column (Series)
df[["price", "quantity"]] # multiple columns (DataFrame)
df[df["price"] > 100]     # filter rows
df.loc[df["region"] == "EU", "price"] # rows + columns
```

```
# Transforming
df["total"] = df["price"] * df["quantity"]
df.groupby("region")["total"].sum()    # GROUP BY in dataframe form
df.sort_values("total", ascending=False).head(10)

# Output
df.to_csv("out.csv", index=False)
df.to_parquet("out.parquet")          # much better for large data
```

8.3 Pandas anti-patterns

⚠ Anti-pattern

for index, row in df.iterrows(): ... as the default way to process rows. iterrows is the slowest possible way to use pandas. Vectorize with column operations (df["a"] + df["b"]), apply() for row-level transforms that can't be vectorized, or numpy directly. iterrows is the line that quietly makes scripts 1000x slower than they need to be.

⚠ Anti-pattern

Chained assignment with inplace=True. df[df.region == "EU"].price.fillna(0, inplace=True) silently fails to modify the original DataFrame in many cases — and inplace=True is being deprecated across pandas anyway. Assign the result back explicitly: df.loc[df.region == "EU", "price"] = df.loc[df.region == "EU", "price"].fillna(0).

⚠ Anti-pattern

Reading the whole CSV when you only need part of it. pd.read_csv has nrows, usecols, dtype, and chunksize parameters. A 5GB CSV opened naively will eat your RAM; opened with chunksize=100_000, it streams.

✓ Practical tip

Use parquet over CSV for anything beyond casual data exchange. Parquet is columnar, typed, compressed, and roughly 10-100x faster to read. CSV has its place for human-readable interchange; for working data, parquet wins.

8.4 Beyond pandas: polars and duckdb

Two newer tools that you'll meet increasingly:

Tool	What it is	When to reach for
polars	Rust-based DataFrame, lazily evaluated	Big data that's too slow in pandas; new projects from scratch

Tool	What it is	When to reach for
duckdb	In-process analytical SQL database	When you want to write SQL against files (parquet, csv) without setting up a database
dask	Distributed/lazy pandas-like API	Truly huge data across machines (less common now polars exists)
pyarrow	Apache Arrow bindings	Foundation under polars, duckdb, modern pandas

In 2026

For datasets that fit in memory and you want fast: polars. For ad-hoc analytics on files or databases without setting up infrastructure: duckdb. For the broadest ecosystem and notebook-friendly exploration: pandas. All three coexist happily in the same project.

8.5 Notebooks: Jupyter and modern alternatives

A notebook is an interactive document mixing code, output, and prose. The dominant tool is Jupyter (and its modern variant JupyterLab). VS Code and Cursor both support notebooks natively now. Notebooks are where data exploration lives.

Tool	Notes	
Jupyter / JupyterLab	The classic	
VS Code / Cursor notebooks	Built into your editor, good for AI-assisted work	
marimo	Reactive notebooks (cells re-run on dependency change)	
Google Colab	Hosted Jupyter with free GPUs	

⚠ Anti-pattern

Notebooks as production code. Notebooks are exploration tools. The cells execute in any order; state persists invisibly; sharing them via git is painful (output gets diffed); they don't get tested. Once code is doing real work, extract it into .py files, import them from notebooks during exploration, and ship the .py files.

8.6 Visualization

Library	Style	Best for
matplotlib	Low-level, total control	Everything; foundation under others
seaborn	Statistical plots on top of matplotlib	Quick exploration plots

Library	Style	Best for
plotly	Interactive HTML plots	Dashboards, notebooks, web embedding
altair	Declarative grammar of graphics	Reproducible, composable visuals
bokeh	Interactive plots	Dashboards

8.7 Data validation and cleanup

Real-world data is messy. The tools you'll lean on:

- Pydantic — schema validation for objects.
- Pandera — pandas-aware schema validation.
- Great Expectations — full data quality / testing framework for pipelines.

✓ Practical tip

For any dataset that enters your code from outside, define a Pydantic or Pandera schema for it. The schema catches malformed rows early; trying to debug pandas operations on bad data is far harder than catching the bad data on entry.

Part 9 — AI, ML, and LLM Work

Python is the de facto language for AI and machine learning. This part is a tour of the landscape — enough to read AI codebases, recognize libraries by name, and direct an agent building AI features.

9.1 The three layers of AI work

Layer	What it means
Classical ML	Tabular data, structured problems. scikit-learn, XGBoost, LightGBM.
Deep learning	Neural networks, custom models. PyTorch, JAX, TensorFlow.
LLM application work	Calling existing models via API. OpenAI, Anthropic, LangChain, LlamaIndex.

Most teams today are at layer three — using pre-trained LLMs as a service rather than training their own. That's where most AI-native product work lives, and what this part focuses on most.

9.2 scikit-learn — classical ML in one library

scikit-learn provides classification, regression, clustering, and the preprocessing/evaluation infrastructure to do them well. The API is uniformly clean.

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

preds = model.predict(X_test)
accuracy_score(y_test, preds)
```

Every scikit-learn model has the same shape

`model.fit(X, y)` to train, `model.predict(X)` to use, `model.score(X, y)` to evaluate. Once you know the pattern, you can swap algorithms in one line. Reach for scikit-learn for structured/tabular problems before considering deep learning.

9.3 PyTorch — the deep learning standard

PyTorch is the dominant framework for neural networks and most published research. Almost every modern model — language, vision, speech — is implemented in PyTorch. TensorFlow still exists; new work overwhelmingly happens in PyTorch.

```
import torch
import torch.nn as nn

class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        return self.fc2(x)

model = SimpleNet()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()
```

You usually won't train models from scratch. The common pattern in product work is downloading a pre-trained model (from Hugging Face) and fine-tuning it on your data, or using it as-is.

9.4 Hugging Face — the model hub

Hugging Face is the GitHub of AI models. The transformers library gives you a uniform way to load and run thousands of pre-trained models — text, image, audio.

```
from transformers import pipeline

# Sentiment analysis – three lines
classifier = pipeline("sentiment-analysis")
classifier("I love this product!")
# [{'label': 'POSITIVE', 'score': 0.9998}]

# Image generation, translation, summarization, embeddings –
# all the same pipeline pattern.
```

Related libraries to recognize: datasets (load standard ML datasets), evaluate (standard metrics), accelerate (multi-GPU made easy), peft (parameter-efficient fine-tuning like LoRA).

9.5 Calling LLMs as a service

The most common AI work in product teams today: call a hosted LLM (Claude, GPT, Gemini, Llama) via API. The SDKs:

```
# Anthropic
from anthropic import Anthropic
client = Anthropic()
```



```
message = client.messages.create(
    model="claude-opus-4-7",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Summarize this email: ..."}],
)
print(message.content[0].text)

# OpenAI
from openai import OpenAI
client = OpenAI()
response = client.chat.completions.create(
    model="gpt-5",
    messages=[{"role": "user", "content": "Summarize this email: ..."}],
)
print(response.choices[0].message.content)
```

⚠ Anti-pattern

Hardcoding API keys in source code or notebooks: `client = Anthropic(api_key="sk-ant-...")`. The SDKs read keys from environment variables by default (`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`) — let them. A leaked key in a git history can rack up thousands of dollars in charges before you notice.

9.6 Streaming responses

For chat UIs and long generations, streaming the tokens as they arrive is the standard pattern.

```
with client.messages.stream(
    model="claude-opus-4-7",
    max_tokens=1024,
    messages=[{"role": "user", "content": prompt}],
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)
```

✓ Practical tip

Combine streaming with FastAPI's `StreamingResponse` to pipe LLM output through your backend to the React frontend (which can read it via Server-Sent Events — see the React doc Part 6.3). This is the standard pattern for any chat-like AI feature.

9.7 Structured outputs

Asking an LLM to return JSON in a specific shape is foundational to product work. Modern SDKs support this directly:

```
from pydantic import BaseModel

class ExtractedInfo(BaseModel):
    name: str
```

```
email: str
intent: str

# OpenAI (parsed API)
result = client.responses.parse(
    model="gpt-5",
    input=email_body,
    text_format=ExtractedInfo,
)
info: ExtractedInfo = result.output_parsed
```

The pattern that works

Define a Pydantic model for what you want back, pass it to the SDK's structured-output mode, get a fully-typed Pydantic object. No string parsing, no JSON repair, no "please reply in this exact format" prompt gymnastics. Available on both major SDKs, and a massive quality and reliability win.

9.8 Embeddings and vector search

An embedding is a vector (list of floats) that represents the meaning of text. Similar text gets similar vectors; you find related content by finding nearby vectors. This is the foundation of semantic search and RAG.

```
# OpenAI embedding
embedding = client.embeddings.create(
    model="text-embedding-3-small",
    input="What's the refund policy?",
).data[0].embedding  # list of 1536 floats

# Voyage (Anthropic-aligned)
import voyageai
vo = voyageai.Client()
result = vo.embed(["What's the refund policy?"], model="voyage-3")
embedding = result.embeddings[0]
```

Vector databases store these and find nearest neighbors. The names you'll see: Pinecone, Weaviate, Qdrant, Chroma, Milvus, pgvector (Postgres extension).

✓ Practical tip

Start with pgvector if you already have Postgres. It's good enough for most apps and avoids running a separate database. Move to a dedicated vector DB only when you genuinely have scale or feature needs Postgres can't meet.

9.9 RAG — Retrieval-Augmented Generation

RAG is how you make an LLM "know" about your data without retraining. The pattern:

```
# 1. Pre-process (offline, once):
```

```
# Split your docs into chunks → embed each chunk → store in vector DB.

# 2. At query time:
def answer(question: str) -> str:
    # Embed the question
    q_vec = embed(question)

    # Find the most similar chunks
    chunks = vector_db.search(q_vec, top_k=5)

    # Stuff them into the prompt
    context = "\n\n".join(c.text for c in chunks)
    prompt = f"Answer based on context:\n{context}\n\nQuestion: {question}"

    # Ask the LLM
    return llm(prompt)
```

⚠ Anti-pattern

Naive RAG that retrieves five chunks, jams them into the prompt, and hopes for the best. Real systems need: smart chunking (don't break in the middle of a sentence), query rewriting, reranking, citation tracking, and evaluation. AI agents will happily generate naive RAG; ask for the upgrades or use a framework that includes them.

9.10 LangChain, LlamaIndex, and the framework question

Frameworks for LLM application work:

Framework	Style	Notes
LangChain	Comprehensive, many integrations	Most popular; complex; APIs have churned
LlamaIndex	Focused on RAG and data ingestion	Cleaner if RAG is the main use case
Haystack	Modular pipelines	Strong on enterprise search
Pydantic AI / Instructor	Lightweight structured outputs	Small, sharp, modern
DSPy	Programmatic LLM workflows with auto-tuning	Research-heavy; promising
Anthropic / OpenAI SDKs directly	No framework	Often the right answer

Honest assessment

For straightforward LLM features — chat, structured extraction, simple RAG — calling the SDKs directly is often cleaner than a framework. Frameworks earn their keep when you need many integrations (lots of model providers, tool calls, data sources) or you're building complex agentic workflows. Start with the SDKs; reach for a framework when you feel the lack.

9.11 Agents and tool use

"Agent" in this context means an LLM that can call functions you provide — search a database, send an email, run a calculation, call another API. The pattern:

1. Define your tools as functions with JSON schemas describing inputs.
2. Pass them to the LLM along with a user request.
3. The LLM responds either with text or with a tool call request.
4. On a tool call, you execute the function and pass the result back.
5. Loop until the LLM produces a final text response.

Both Anthropic and OpenAI SDKs support tool use natively. LangChain and LlamaIndex wrap this in their own abstractions. MCP (Model Context Protocol) is becoming the standard way to expose tools across model providers.

9.12 Evaluating LLM features

LLM outputs aren't deterministic. You can't just write a unit test asserting the output. The names to recognize:

Tool	Style	
pytest + LLM-as-judge	Use an LLM to grade outputs	
Promptfoo	Declarative prompt tests	
Langfuse / LangSmith / Phoenix	Tracing + eval platforms	
Braintrust	Eval-focused platform	
Inspect AI	Anthropic's eval framework	

✓ **Practical tip**

Before you ship an LLM feature, write 20-50 representative example inputs and the outputs you expect. Run them on every model/prompt change. Without this, you have no way to know if a "small prompt tweak" silently broke half your users' use cases. AI-assisted product work without evals is shipping blind.

9.13 Watch-outs for AI-written AI code

- Outdated SDK syntax — OpenAI's library has had several major versions; agents sometimes mix them up.
- Excessive use of LangChain when the SDK alone would do.
- RAG with no chunking strategy, no reranking, no evaluation.
- Embeddings re-computed on every request instead of cached.
- Whole files dumped into the prompt instead of relevant chunks.
- API keys in code, prompts logged with PII.
- No retries, no timeouts, no rate limiting on LLM calls.
- Synchronous LLM calls inside async code (use the async clients).

Part 10 — Testing

For testing philosophy, what to test, and the role of tests in AI-assisted development — see the React & TypeScript Fundamentals doc, Part 8. This section covers the Python-specific mechanics.

10.1 pytest — the standard

pytest is Python's testing tool. There's also a built-in unittest module — you can ignore it for new code; pytest is shorter, smarter, and supports unittest tests if you encounter them.

```
# tests/test_users.py
from app.users import normalize_email

def test_normalize_email_lowercases():
    assert normalize_email("MAYA@example.com") == "maya@example.com"

def test_normalize_email_strips_whitespace():
    assert normalize_email("  maya@example.com  ") == "maya@example.com"
```

Run with: `pytest`, or `pytest tests/test_users.py` for one file.

10.2 Fixtures — setup and teardown

```
import pytest

@pytest.fixture
def db():
    conn = create_test_db()
    yield conn
    conn.close()                # teardown

def test_create_user(db):       # pytest provides the fixture
    user = create_user(db, name="Maya")
    assert user.id is not None
```

Fixtures eliminate setup duplication and handle cleanup. They can build on each other — a request fixture might depend on a db fixture, which depends on a settings fixture.

10.3 Parametrize — one test, many cases

```
@pytest.mark.parametrize("email,expected", [
    ("MAYA@example.com", "maya@example.com"),
    ("  maya@example.com  ", "maya@example.com"),
    ("Maya+sub@Example.COM", "maya+sub@example.com"),
])
def test_normalize_email(email, expected):
    assert normalize_email(email) == expected
```

10.4 Mocking

```
from unittest.mock import patch

def test_sends_email(monkeypatch):
    sent = []
    def fake_send(to, subject, body):
        sent.append((to, subject, body))
    monkeypatch.setattr("app.mailer.send", fake_send)

    notify_user("maya@example.com")
    assert sent == [("maya@example.com", "Welcome", "...")]
```

pytest's monkeypatch and unittest.mock cover most cases. For HTTP, respx (httpx mocking) or pytest-httpx is cleaner than mocking by hand.

10.5 Testing FastAPI

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_get_user():
    response = client.get("/users/1")
    assert response.status_code == 200
    assert response.json()["name"] == "Maya"
```

10.6 Coverage

pytest-cov measures how much of your code your tests exercise.

```
pytest --cov=app --cov-report=term-missing
```

⚠ Anti-pattern

Chasing 100% coverage as a goal. High coverage is correlated with quality, but 100% is wasteful and incentivizes meaningless tests. 70-85% on application code is a sensible bar. Critical paths (auth, payments, data integrity) should be near 100%; UI glue code can be lower.

10.7 Python-specific test ideas worth requesting

- Property-based tests with hypothesis — describes what should be true and generates random inputs to try to break it.
- Snapshot tests with syrupy — for output that's tedious to assert by hand.
- Schema tests on API responses — Pydantic-validate the response shape in addition to specific values.

Part 11 — Production Concerns

Folder structure, environment variables, monitoring, deployment, and CI/CD overlap heavily with the React doc — see Parts 9 and 10 of the React & TypeScript Fundamentals. This section focuses on Python-specific production concerns.

11.1 Project layout

```
my-app/  
  pyproject.toml  
  README.md  
  .env.example  
  src/  
    my_app/  
      __init__.py  
      main.py  
      api/  
      models/  
      services/  
      db/  
  tests/  
    test_users.py  
    test_api.py  
  alembic/           # if using SQLAlchemy migrations  
  scripts/           # one-off and ops scripts
```

✓ Practical tip

Put your package under `src/`. The `src/` layout prevents you from accidentally importing your package via the current directory instead of the installed one — a subtle gotcha that produces "works in dev, breaks in prod" bugs. Modern Python tooling defaults to this layout for good reason.

11.2 Configuration

Use `pydantic-settings` for typed environment configuration:

```
from pydantic_settings import BaseSettings  
  
class Settings(BaseSettings):  
    database_url: str  
    openai_api_key: str  
    log_level: str = "INFO"  
  
    class Config:  
        env_file = ".env"  
  
settings = Settings() # raises if required vars missing
```


Now every required env var is documented and validated at startup. Missing or malformed config fails loud and early, not three hours into operation when the first user hits the broken endpoint.

11.3 Logging in production

Use structlog or the standard logging module with JSON formatting. Structured logs are searchable and queryable in log aggregators; plain text is not.

```
import structlog
log = structlog.get_logger()

log.info("user_signed_up", user_id=user.id, source="web")
# Outputs: {"event": "user_signed_up", "user_id": 123, "source": "web", ...}
```

11.4 Packaging Python for deployment

Most production Python deploys as a container — a Docker image with Python, your dependencies, and your code. The pattern:

```
# Dockerfile (sketch)
FROM python:3.13-slim
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen
COPY src/ ./src/
CMD ["uv", "run", "uvicorn", "my_app.main:app", "--host", "0.0.0.0"]
```

Where to host

Same options as in the React doc Part 10.8 — Render, Railway, Fly.io for simple container hosting; AWS/GCP/Azure for full cloud; Vercel Functions and AWS Lambda for serverless (with caveats around Python cold starts and package sizes — AWS Lambda + Mangum is a common pattern for FastAPI). Hugging Face Spaces and Modal are popular for ML-heavy workloads.

11.5 ASGI servers

FastAPI itself doesn't serve HTTP — it produces an ASGI application that an ASGI server runs. The choices:

- uvicorn — the standard, used in nearly every FastAPI tutorial.
- hypercorn — supports HTTP/2 and HTTP/3.
- gunicorn + uvicorn workers — common production setup with multiple workers.
- granian — newer Rust-based ASGI server, very fast.

11.6 Performance gotchas

⚠ Anti-pattern

Blocking I/O inside FastAPI async endpoints. `requests.get()`, opening files without `aiofiles`, slow sync database calls — these block the event loop, stalling all other requests. Either use async libraries throughout, or move the blocking call to a thread pool with `anyio.to_thread.run_sync()`. FastAPI's perf advantage disappears the moment one sync blocker sneaks into an async handler.

⚠ Anti-pattern

N+1 queries — fetching a list, then making one database query per item. Common with ORMs. Use eager loading (SQLAlchemy's `joinedload` or `selectinload`) or batch fetching. APIs that feel slow are almost always doing this.

Part 12 — Working with AI Agents in Python

General AI agent workflow advice is in the React doc, Part 11. This section is the Python-specific layer — what's different about agents writing Python.

12.1 What agents do well in Python

- Glue code, scripts, automation — Python's expressiveness shines, and there's enormous public training data.
- Data manipulation with pandas/numpy — extremely common in training data.
- FastAPI endpoints — well-established patterns.
- LLM SDK usage — SDKs are heavily documented and recent.
- pytest — uniform conventions.

12.2 What agents struggle with in Python

- Async correctness — mixing sync and async, forgotten awaits, blocking calls in async handlers.
- Modern syntax — they still sometimes write Python 3.6-era code (`str.format()`, `Dict` from `typing` instead of `dict[]`).
- Type hints — often missing on private functions or for return types.
- Library version skew — particularly OpenAI SDK v0 vs v1, SQLAlchemy 1.x vs 2.x, Pydantic 1.x vs 2.x. Each had a breaking rewrite; agents sometimes mix syntax from both.
- File system assumptions — using `__file__` relative paths or assuming the CWD.
- Pythonic idioms — they'll write `for i in range(len(items))` instead of `for item in items`, or build dicts in a loop instead of using a comprehension.

12.3 Setting up a Python project for AI agents

Same idea as the React doc Part 11.2, applied to Python:

- **CLAUDE.md / AGENTS.md** at the repo root. Document Python version, package manager (uv recommended), framework (FastAPI), ORM, validation library (Pydantic), test runner, and code conventions.
- **Pyright in strict mode** or mypy with strict flags. Type errors are the agent's fastest feedback signal.
- **Ruff configured** with sensible rules. AI agents respect the configured ruleset.
- **pytest set up with quick smoke tests** the agent can run after every change.
- **Pre-commit hooks** running ruff, pyright, and pytest before commits land.
- **A dev script** (make dev or just dev) that spins up the app reproducibly.

12.4 The Python review checklist

When an agent shows you a Python diff, scan for:

1. Imports — does it import from typing.List or use dict[] / list[] (modern)? Does it import something that's unused?
2. Type hints — are functions annotated? Is anything Any that shouldn't be?
3. Mutable defaults — def foo(x=[]) is broken.
4. Bare except clauses — both except: and except Exception: pass are smells.
5. Hardcoded secrets — API keys, database URLs, credentials in source.
6. SQL injection — f-strings in queries, string concatenation in raw SQL.
7. Resource leaks — files opened without with, database sessions not closed.
8. N+1 queries — loops with a database call inside.
9. Print statements — should be logging in anything beyond a script.
10. Async correctness — every await present? Any blocking I/O in async code? Any forgotten async def?
11. Pydantic / Validation — does any external data cross the boundary unvalidated?
12. Pinned dependencies — does pyproject.toml have loose versions for security-critical libs?
13. Tests — does the change include pytest tests with at least one failure path?

12.5 A good Python prompt

Bad: "Add a user signup endpoint."

Good: "Add a POST /users endpoint to src/my_app/api/users.py. Use a Pydantic UserCreate model (name: str, email: EmailStr, password: SecretStr min length 8). On success, hash the password with passlib (bcrypt), insert via the existing UserRepository in src/my_app/db/users.py, and return a Pydantic UserOut model (no password fields). On a duplicate email, return 409. On a validation error, FastAPI's default 422. Add pytest tests in tests/test_users.py covering: happy path, duplicate email, short password, invalid email. Use the existing TestClient fixture in tests/conftest.py."

12.6 The verification loop, Python edition

Mirroring the React doc, but with Python tools:

1. ruff check . passes
2. ruff format --check . passes
3. pyright src/ passes
4. pytest passes
5. uvicorn or your dev server still runs

6. The new feature works in the browser or via curl
7. At least one error case works as designed

If your agent can run all of these itself, ask it to before declaring the task done. If it can't, do it yourself before accepting the change.

12.7 The single best Python habit

Open a Python REPL — `python` or, better, `ipython` — and explore the code the AI wrote. Import the function. Call it with different arguments. Check the types. See what happens with the empty list, the `None`, the malformed input. Python's REPL is the fastest debugger and the best way to convince yourself code does what it claims to do.

REPL > reading

Five minutes in a REPL teaches you more about a piece of Python code than thirty minutes of reading it. Make REPL exploration a routine after non-trivial AI changes. In a notebook environment, this is the same habit — just inline.

Glossary

Quick reference for Python-specific terms. For cross-cutting terms (HTTP, REST, OAuth, CI/CD, etc.) see the React & TypeScript Fundamentals doc Glossary.

aiohttp	Async HTTP library, both client and server. Older than httpx.
Alembic	Schema migrations for SQLAlchemy.
Anaconda	Distribution of Python with many ML/data libraries pre-installed. Common in data science.
Anthropic SDK	Official Python client for Claude. PyPI: anthropic.
argparse	Standard library tool for parsing command-line arguments.
asgiref	Foundation library for ASGI applications.
ASGI	Asynchronous Server Gateway Interface — the async successor to WSGI. FastAPI, Starlette.
asyncio	Standard library async runtime.
Awaitable	Anything you can await — usually a coroutine.
bcrypt / argon2	Password hashing algorithms; use a library, never DIY.
BaseModel	Pydantic's base class for validated data models.
black	Opinionated code formatter. Now largely subsumed by ruff format.
Celery	Distributed task queue, the standard for background jobs in Python.
click / typer	Libraries for building command-line interfaces. typer is FastAPI-style.
Conda	Package manager that handles non-Python dependencies; popular in ML.
Context manager	Object usable in a with statement; defines <code>__enter__</code> and <code>__exit__</code> .
Coroutine	An async function, after being called. Must be awaited or scheduled.
CPython	The standard Python implementation, written in C.
Dataclass	Decorator that generates <code>__init__</code> , <code>__repr__</code> , <code>__eq__</code> from class fields.
Decorator	Function that wraps another function to modify its behavior. Uses <code>@</code> syntax.
Django	Batteries-included web framework. ORM, admin, auth, templates.
Docstring	String literal at the top of a function/class/module. Documentation.
duck typing	If it walks like a duck... checking behavior, not type.
Dunder method	Method with double-underscore name (<code>__init__</code> , <code>__str__</code>). Special protocol methods.
EmailStr	Pydantic type with built-in email validation.
embedding	Vector representation of meaning, produced by an embedding model.

enum.Enum	Standard library for grouped constants.
f-string	Modern string formatting: f"Hello, {name}". Preferred over .format() and %.
FastAPI	Modern async-first Python web framework with Pydantic integration.
Flask	Minimal web framework, sync-first.
functools	Standard library: lru_cache, partial, reduce, wraps.
generator	Function with yield; lazy iterator producing values on demand.
GIL	Global Interpreter Lock. One thread of Python bytecode at a time in CPython.
gunicorn	Process manager / WSGI server, often paired with uvicorn workers.
Hugging Face	Hub for AI models. transformers, datasets, accelerate libraries.
httpx	Modern HTTP client, sync + async. Replaces requests.
hypothesis	Property-based testing library.
IPython	Enhanced interactive Python shell. Foundation of Jupyter.
JIT	Just-In-Time compilation. PyPy has a Python JIT; CPython 3.13+ has tier-2 JIT.
JSON Schema	Standard for describing JSON structure. Pydantic generates it.
Jupyter / JupyterLab	Notebook interface for interactive Python.
LangChain	LLM application framework. Many integrations; complex.
LangGraph	Graph-based agent workflow library, by LangChain authors.
LlamaIndex	RAG-focused framework, alternative to LangChain.
lru_cache	Decorator that caches function results based on arguments.
marimo	Modern reactive notebook environment.
matplotlib	Foundational plotting library.
MCP	Model Context Protocol. Standard for exposing tools and resources to LLMs.
multiprocessing	Standard library for parallel processes; sidesteps the GIL.
mypy	Static type checker for Python.
NumPy	Array library, foundation of the Python data ecosystem.
OpenAI SDK	Official Python client for GPT models. PyPI: openai.
ORM	Object-Relational Mapper. SQLAlchemy, Django ORM, Tortoise.
pandas	DataFrames and tabular data manipulation.
pathlib	Modern object-oriented file path handling. Replaces os.path.
Pydantic	Runtime data validation using type hints. v2 is dramatically faster than v1.

pydantic-settings	Pydantic-based environment configuration.
pyenv	Manage multiple Python versions on one machine.
pyright	Microsoft's type checker, powers Pylance in VS Code. Faster than mypy.
pip	Standard package installer.
pipx	Install standalone Python apps in isolated envs.
polars	Rust-based DataFrame library, fast alternative to pandas.
poetry	Project + dependency manager. Older default; uv now widely preferred.
pyproject.toml	Standard project metadata file.
pytest	Standard testing framework.
PyTorch	Dominant deep-learning framework.
RAG	Retrieval-Augmented Generation. Fetch relevant chunks → stuff into LLM prompt.
repl	Read-Eval-Print Loop. The Python interactive shell.
requests	Older HTTP library. Sync only. Replaced by httpx in new code.
ruff	Rust-based linter + formatter. Replaces flake8, black, isort, and more.
scikit-learn	Classical machine learning library. Uniform fit/predict API.
scipy	Scientific computing on top of numpy.
SecretStr	Pydantic type that hides its value in repr/logs.
seaborn	Statistical plotting on top of matplotlib.
slots	<code>__slots__</code> — limits a class's attributes; saves memory.
SQLAlchemy	Most popular Python ORM and query builder. v2 is current.
SQLModel	Pydantic-style ORM on top of SQLAlchemy, from FastAPI's author.
Starlette	ASGI framework. Foundation under FastAPI.
streamlit	Build simple data UIs in pure Python. Popular for ML demos.
structlog	Structured logging library.
tox / nox	Run tests across multiple Python versions.
transformers	Hugging Face library for using pre-trained models.
TensorFlow	Google's deep-learning framework. Older; new work mostly happens in PyTorch.
TypedDict	typing module construct for dicts with known keys and types.
typer	FastAPI-style CLI library.
uv	Rust-based all-in-one Python toolchain. Replaces pip, venv, pyenv, poetry.
uvicorn	ASGI server, the standard for running FastAPI/Starlette in dev and prod.

venv	Virtual environment — isolated Python install per project.
Vector DB	Database for nearest-neighbor search on embeddings. Pinecone, Qdrant, pgvector.
voyageai	Embedding models (Anthropic-aligned). Often paired with Claude.
WSGI	Older sync gateway interface. Flask, Django historically; ASGI is the modern shape.
XGBoost / LightGBM	Gradient-boosting libraries for tabular ML.
yield	Keyword that turns a function into a generator.

Appendix — Command Cheat Sheet

Python commands you'll meet constantly when working with AI agents.

Project setup with uv (recommended)

```
# Create a new project
uv init my-app
cd my-app

# Add dependencies
uv add fastapi sqlalchemy pydantic
uv add --dev pytest ruff pyright

# Run a command in the venv
uv run uvicorn my_app.main:app --reload
uv run pytest
uv run python script.py
```

Project setup with pip + venv (traditional)

```
# Create a venv
python -m venv .venv
source .venv/bin/activate      # macOS/Linux
.venv\Scripts\activate        # Windows

# Install + freeze
pip install fastapi sqlalchemy pydantic
pip install -r requirements.txt
pip freeze > requirements.txt
```

Daily workflow

```
ruff check .          # lint
ruff check . --fix    # lint + auto-fix
ruff format .         # format
pyright src/          # type check
pytest                # run tests
pytest -k test_users  # only matching tests
pytest -x             # stop at first failure
pytest --cov=app       # with coverage

python -m my_app       # run a package as a script
ipython               # better REPL
python -m pdb script.py # debug a script
```

FastAPI dev

```
uvicorn my_app.main:app --reload
uvicorn my_app.main:app --host 0.0.0.0 --port 8000

# Visit /docs for interactive API documentation
```

Useful one-liners

```
# Show Python version + venv
python --version
which python

# Pretty-print JSON
echo '{"a":1}' | python -m json.tool

# Run a quick HTTP server in the current dir
python -m http.server 8000

# Time a snippet
python -m timeit -s "data = list(range(1000))" "sum(data)"
```

Closing Thought

Python's surface area is bigger than React + TypeScript's. The language itself is small, but it sits at the center of so much — backends, data, ML, AI, scripting — that what counts as "Python fluency" varies by project.

The mental models that travel are the ones in Parts 1–3: the language, the object model, type hints and Pydantic. From there you bolt on whichever stack the project needs. FastAPI today, pandas tomorrow, LangChain next month. The fundamentals stay; the ecosystem keeps moving.

Pair this doc with the React & TypeScript Fundamentals doc, and you have most of what you need to direct AI agents across a modern full-stack AI-native project. The rest is reps.